Introducción Práctica a GitHub Actions: Integración Continua con Python, pytest y Pull Requests en el Caso StockGuard

Smith Valencia
Facultad de Ingeniería, Universidad de Cundinamarca
Big Data
Ing. Oscar Jobany Gómez Ochoa
29 de octubre de 2026

Introducción Práctica a GitHub Actions: Integración Continua con Python, pytest y Pull Requests en el Caso StockGuard

La integración continua automatiza verificaciones cada vez que un equipo integra cambios en un repositorio. GitHub Actions es la plataforma con la que GitHub permite definir esos procesos como *workflows* escritos en YAML, que se ejecutan en máquinas llamadas *runners* ante eventos como un *push* o un *pull request*.

Este documento presenta el desarrollo de la guía de trabajo autónomo *Introducción práctica a GitHub Actions*, cuyo caso de práctica es StockGuard, un clasificador del nivel de riesgo de inventario de un supermercado. Con capturas de pantalla propias se documentan la creación del repositorio, la implementación de la regla de negocio, las pruebas con pytest, la configuración del workflow, una falla controlada y su corrección, un Pull Request y una ejecución manual. Al final se responden las diez preguntas de análisis de la sección 17 de la guía.

Desarrollo de la Práctica

Preparación del Repositorio y Estructura del Proyecto

Se creó el repositorio público *github-actions-stockguard* en GitHub, que inicialmente contenía solo el archivo README.md del commit inicial. Después, en Visual Studio Code, se construyó la estructura propuesta por la guía: la carpeta .github/workflows con ci.yml, el paquete src con __init__.py e inventario.py, la carpeta tests con test_inventario.py y el archivo requirements.txt. Los cambios se registraron con git add, git commit y git push, y el repositorio remoto quedó con las carpetas del proyecto y un archivo .gitignore agregado para no versionar archivos generados.

Figura 1

Repositorio github-actions-stockguard recién creado en GitHub

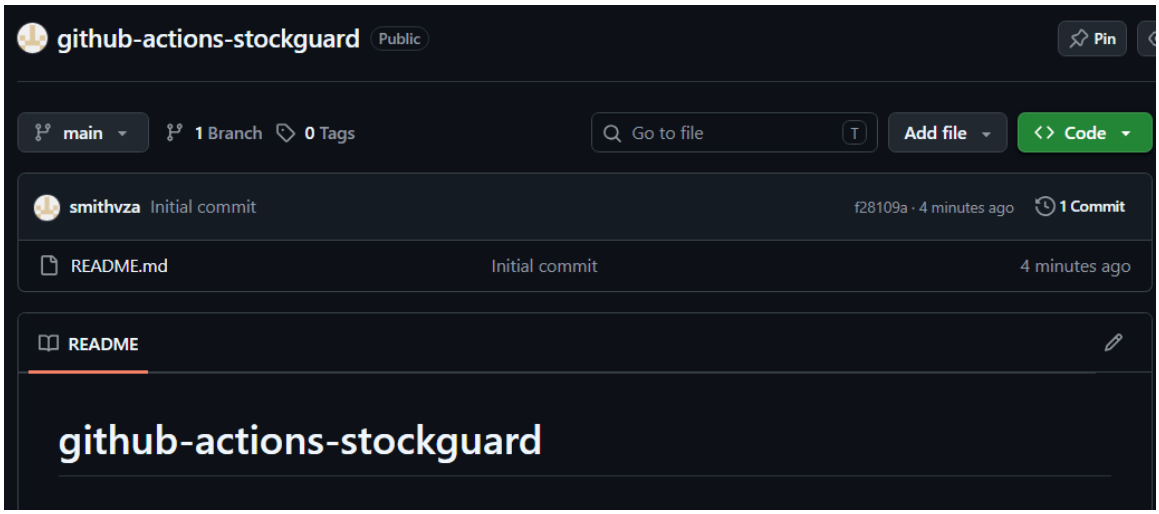


Figura 2

Estructura de carpetas y archivos en Visual Studio Code

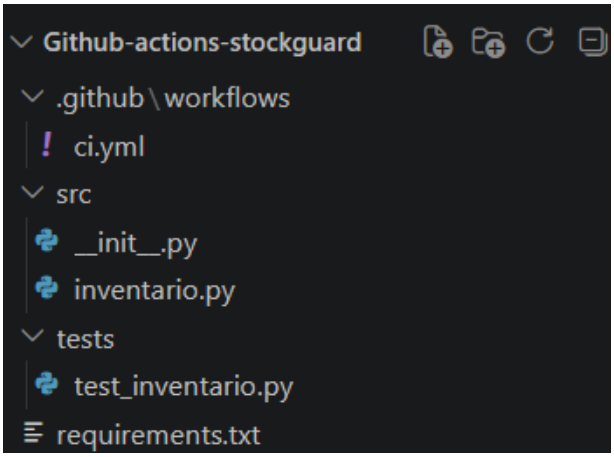


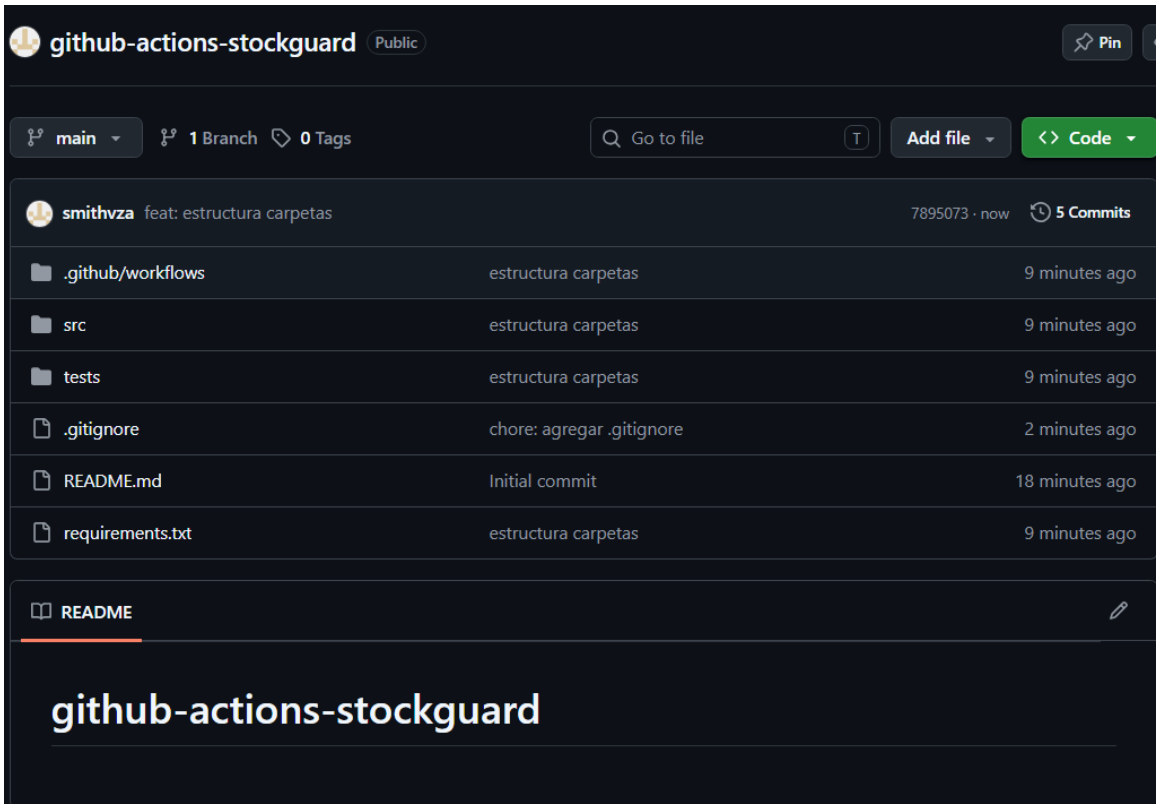
Figura 3

Commit y push de la estructura inicial

```
PS C:\Users\srnit\OneDrive\Desktop\Github-actions-stockguard>  git add .
>>   git commit -m "feat: estructura carpetas"
>>   git push
[main 7895073] feat: estructura carpetas
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 194 bytes | 194.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/smithvza/github-actions-stockguard.git
   342af4d..7895073  main -> main
PS C:\Users\srnit\OneDrive\Desktop\Github-actions-stockguard>
```

Figura 4

Repositorio actualizado en GitHub



Implementación de la Regla de Negocio

En src/inventario.py se implementó la función clasificar_stock, que recibe el stock actual y el stock mínimo y devuelve AGOTADO, CRITICO, BAJO o NORMAL según la regla de la

guía. La función lanza ValueError cuando el stock es negativo o el mínimo no es mayor que cero.

Figura 5

Implementación de la función clasificar_stock

```
src > inventario.py > clasificar_stock
1   def clasificar_stock(stock_actual: int, stock_minimo: int) -> str:
2       if stock_actual < 0:
3           raise ValueError("El stock actual no puede ser negativo")
4       if stock_minimo <= 0:
5           raise ValueError("El stock minimo debe ser mayor que cero")
6       if stock_actual == 0:
7           return "AGOTADO"
8       if stock_actual <= stock_minimo * 0.5:
9           return "CRITICO"
10      if stock_actual <= stock_minimo:
11          return "BAJO"
12      return "NORMAL"
```

Antes de automatizar se realizó la prueba manual solicitada. Con un mínimo de 20 y 7 unidades se esperaba un estado crítico, porque 7 es menor que el 50 % del mínimo. La ejecución en Python lo confirmó al imprimir CRITICO.

Figura 6

Prueba manual de la regla de negocio

```
PS C:\Users\srnit\OneDrive\Desktop\Github-actions-stockguard> python -c "from src.inventario import clasificar_stock;
print(clasificar_stock(7, 20))"
CRITICO
```

Pruebas Automatizadas con pytest

Se escribieron cinco pruebas en tests/test_inventario.py: una por cada estado posible y una para el caso inválido. Para trabajar en un entorno aislado se creó un entorno virtual con python -m venv venv y se activó con Activate.ps1. La dependencia se declaró en requirements.txt y se instaló con pip. También se agregó un archivo pytest.ini que incluye la raíz del proyecto en la ruta de búsqueda de módulos, necesario para poder importar src. Al ejecutar pytest -v las cinco pruebas quedaron en estado PASSED.

Figura 7

Pruebas unitarias en test_inventario.py



Figura 8

Creación y activación del entorno virtual

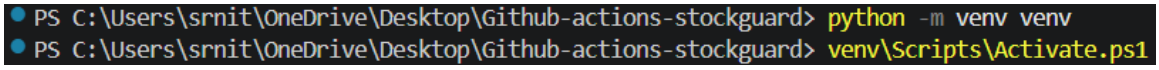


Figura 9

Archivo requirements.txt e instalación de dependencias

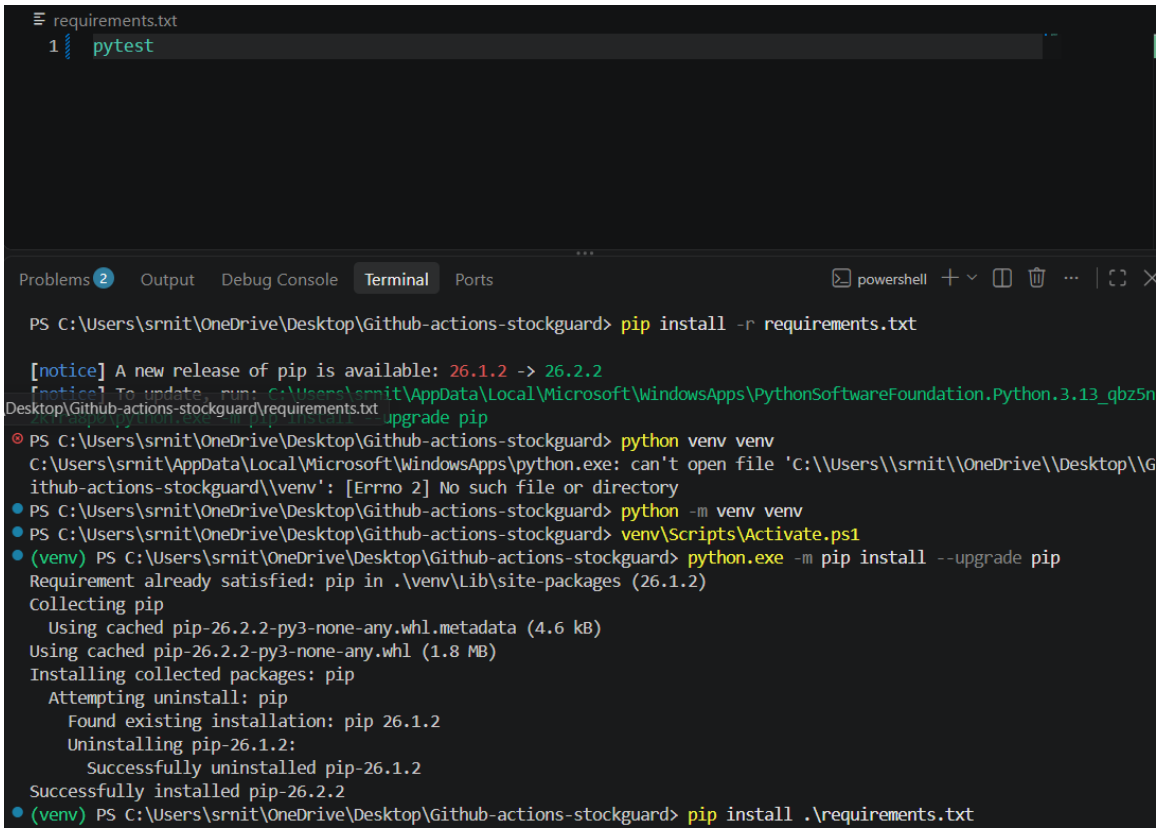
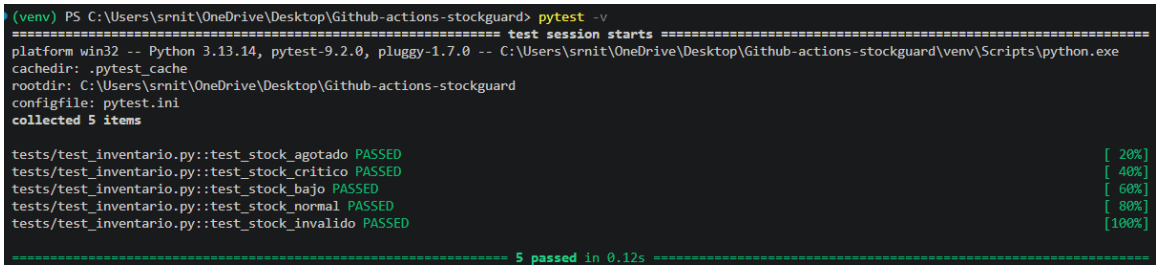


Figura 10

Ejecución local de pytest con cinco pruebas aprobadas



Configuración del Workflow

El archivo ci.yml define el workflow *CI StockGuard*. Su run-name incluye el usuario que dispara cada ejecución mediante github.actor; se activa con push y pull_request hacia main y con workflow_dispatch para ejecuciones manuales. El job validar-python corre en ubuntu-latest y realiza cinco steps: descargar el repositorio, configurar Python 3.12, instalar dependencias,

validar sintaxis con compileall y ejecutar pytest -v, siguiendo el esquema que GitHub documenta para proyectos Python. El archivo se subió al repositorio junto con pytest.ini y requirements.txt.

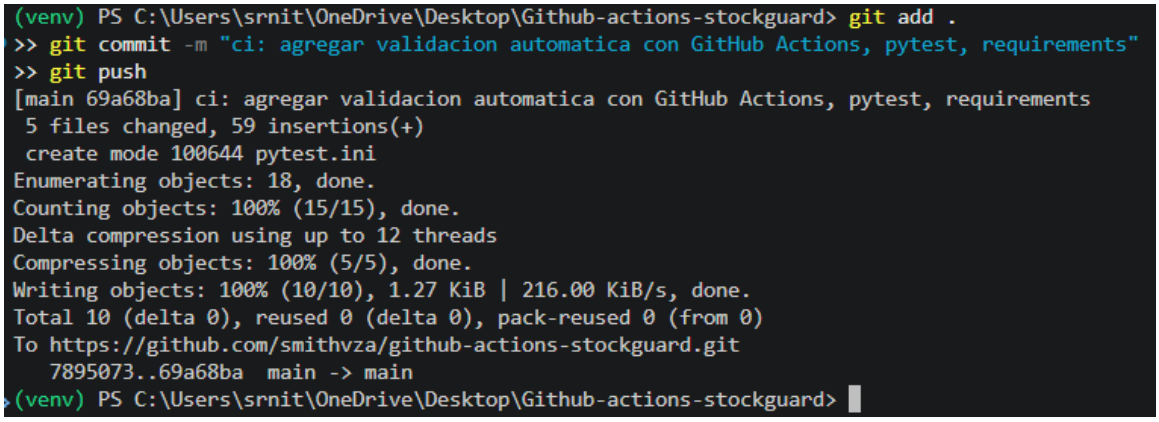
Figura 11

Workflow ci.yml



Figura 12

Commit y push del workflow



Primera Ejecución en GitHub Actions

La ejecución #4, nombrada «CI ejecutado por smithvza», terminó con éxito en 8 segundos y todos sus steps aparecen en verde. Al abrir el log del step *Ejecutar pruebas* se observa que el runner usó Linux, Python 3.12.15 y pytest 9.2.0, y que las cinco pruebas pasaron en 0,01 s.

Figura 13

Ejecución #4 del workflow: todos los steps exitosos

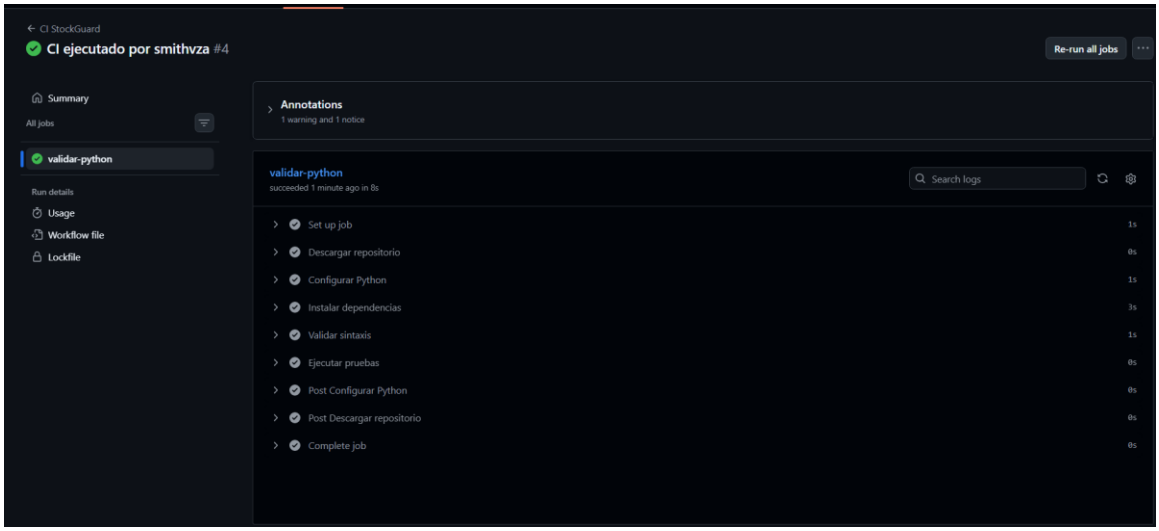
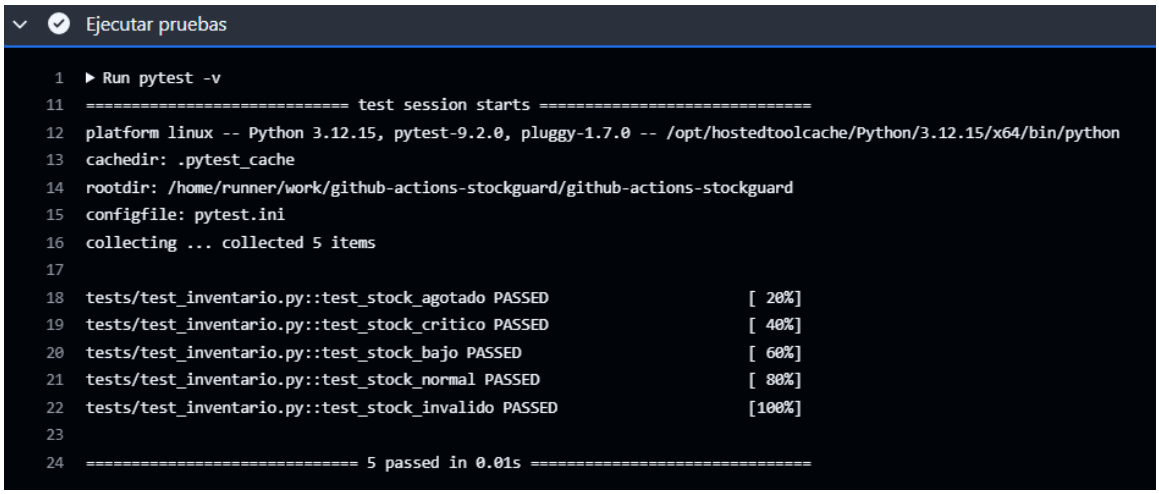


Figura 14

Salida de pytest en el runner de GitHub Actions



Falla Controlada y Restauración

Para provocar una falla, la prueba test_stock_critico se cambió para esperar "NORMAL" en lugar de "CRITICO", y se subió con el mensaje «test: provocar fallo controlado de CI». GitHub Actions generó la ejecución #5, marcada con una X roja, donde falló el step *Ejecutar pruebas*. El log muestra que pytest recolectó cinco pruebas y que la fallida fue test_stock_critico, con el mensaje AssertionError: assert 'CRITICO' == 'NORMAL'.

Figura 15

Modificación intencional de la prueba y commit del fallo controlado

```python
import pytest
from src.inventario import clasificar_stock

def test_stock_agotado():
    assert clasificar_stock(0, 20) == "AGOTADO"

def test_stock_critico():
    assert clasificar_stock(7, 20) == "NORMAL"

def test_stock_bajo():
    assert clasificar_stock(15, 20) == "BAJO"

def test_stock_normal():
    assert clasificar_stock(30, 20) == "NORMAL"

def test_stock_invalido():
    with pytest.raises(ValueError):
        clasificar_stock(-2, 20)
```

```
(venv) PS C:\Users\srnit\OneDrive\Desktop\Github-actions-stockguard> git add .
>> git commit -m "test: provocar fallo controlado de CI"
>> git push
[main d4d8cf6] test: provocar fallo controlado de CI
 1 file changed, 1 insertion(+), 1 deletion(-)
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 370 bytes | 370.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/smithvza/github-actions-stockguard.git
   69a68ba..d4d8cf6  main -> main
```

Figura 16

Ejecución #5 fallida y step Ejecutar pruebas

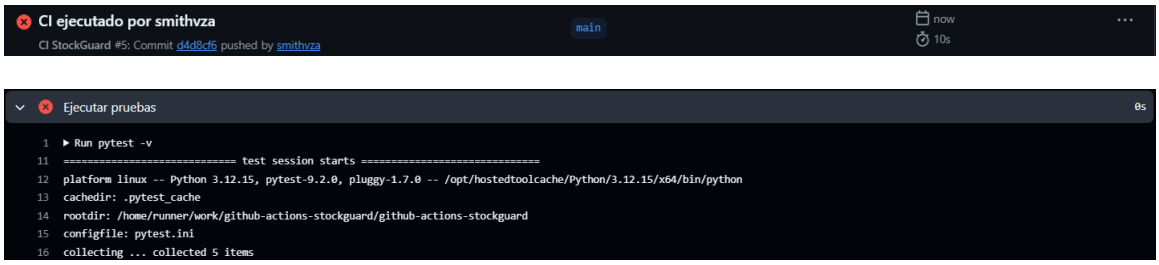


Figura 17

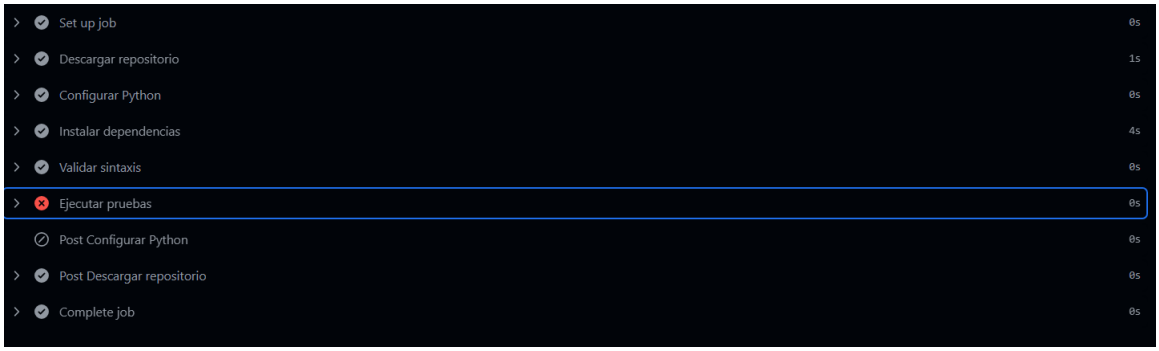
Mensaje de error producido por pytest en el log



Respecto a las preguntas del paso 12 de la guía, el step que falló fue *Ejecutar pruebas*; los steps anteriores terminaron correctamente; y, visualmente, la ejecución se diferencia de la anterior por el ícono rojo en el step fallido y porque el step posterior *Post Configurar Python* fue omitido.

Figura 18

Estado de los steps en la ejecución fallida



Después se restauró el valor correcto de la prueba y se subió el commit «test: restaurar prueba correcta». La ejecución #6 volvió a terminar en verde.

Figura 19

Commit de restauración y ejecución #6 exitosa

```
(venv) PS C:\Users\srnit\OneDrive\Desktop\Github-actions-stockguard> git add .
>> git commit -m "test: restaurar prueba correcta"
>> git push
[main e579f8d] test: restaurar prueba correcta
 1 file changed, 1 insertion(+), 1 deletion(-)
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 364 bytes | 364.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/smithvza/github-actions-stockguard.git
   d4d8cf6..e579f8d  main -> main
```

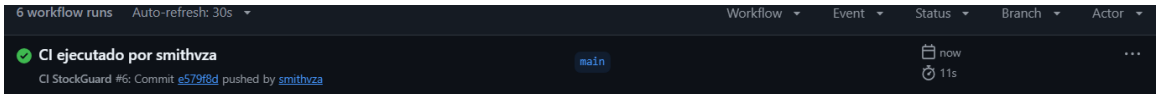


Figura 20

Steps de la ejecución #6 tras la corrección



Pull Request con Rama de Trabajo

Para no trabajar directamente sobre main se creó la rama feature/alerta-sobrestock y en ella se agregó la función detectar_sobrestock, que lanza ValueError ante valores inválidos y devuelve True cuando el stock actual supera al máximo. Luego se abrió el Pull Request #1, «feat: agregar regla de sobrestock», hacia main. El evento pull_request ejecutó el workflow, y el panel del PR muestra que todos los checks pasaron y que no existen conflictos con la rama base.

Figura 21

Creación de la rama feature/alerta-sobrestock

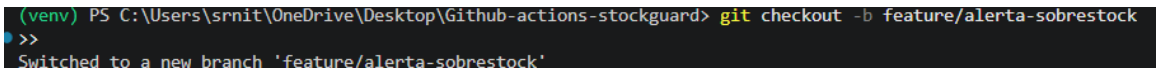


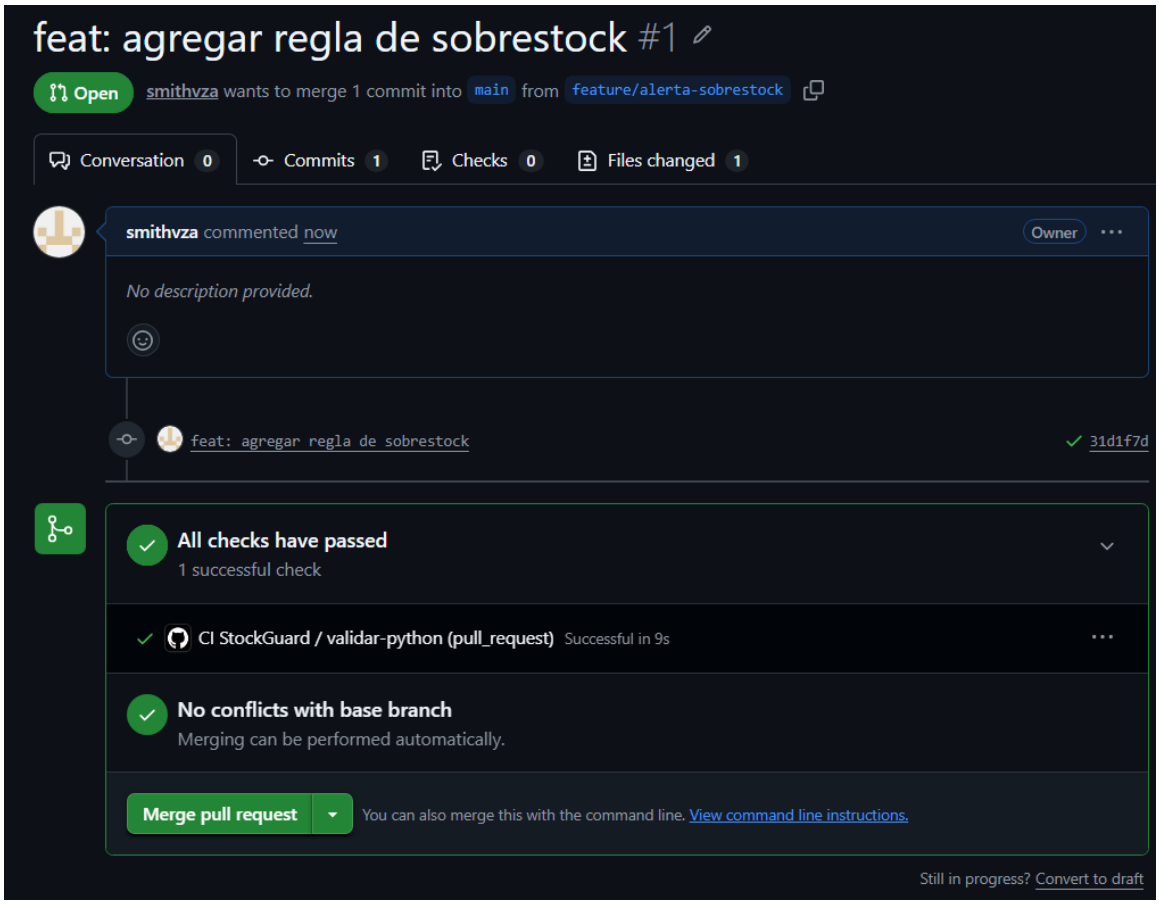
Figura 22

Función detectar_sobrestock

```python
def detectar_sobrestock(stock_actual: int, stock_maximo: int) -> bool:
    if stock_actual < 0 or stock_maximo <= 0:
        raise ValueError("Valores invalidos")
    return stock_actual > stock_maximo
```

Figura 23

Pull Request #1 con los checks de GitHub Actions aprobados



Ejecución Manual del Workflow

Gracias al trigger workflow_dispatch, el workflow puede lanzarse desde la pestaña Actions con el botón *Run workflow*. La ejecución #9 aparece como disparada manualmente por el usuario smithvza sobre el commit 8ba7091 de main, con estado Success y duración de 11 s.

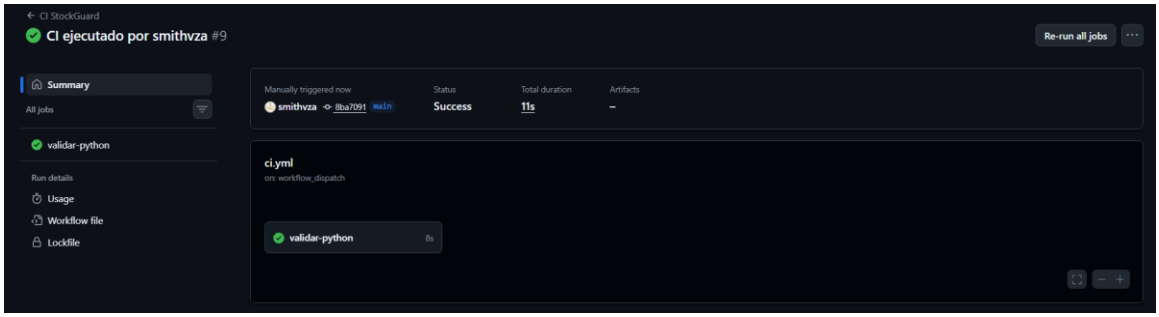
Figura 24

Botón Run workflow en la pestaña Actions



Figura 25

Ejecución #9 disparada manualmente con workflow_dispatch



Respuestas a las Preguntas de Análisis

Pregunta 1

¿Qué ejecución fue exitosa, cuál fue fallida y qué step determinó el resultado?

La ejecución #4 fue exitosa y se originó en el commit 69a68ba, con todos los steps en verde. La ejecución #5 fue fallida y la produjo el commit d4d8cf6, que cambió el valor esperado de una prueba. En ambos casos el step que determinó el resultado fue *Ejecutar pruebas*: en la #4 terminó bien y en la #5 falló, lo que a su vez provocó que el step *Post Configurar Python* se omitiera. La ejecución #6, con el commit e579f8d, volvió a pasar tras restaurar la prueba.

Pregunta 2

¿Por qué ejecutar pytest localmente no reemplaza ejecutar pytest en GitHub Actions?

Porque el entorno local y el del runner son distintos. En esta práctica las pruebas locales corrieron en Windows con Python 3.13.14 dentro de un entorno virtual, mientras que en GitHub corrieron en un runner limpio de Linux con Python 3.12.15. Si una dependencia estuviera instalada en el computador pero no declarada en requirements.txt, o si un archivo necesario, como pytest.ini, no se hubiera subido al repositorio, las pruebas pasarían localmente y fallarían en Actions. Además, la ejecución en Actions ocurre de forma automática y deja evidencia visible para todo el equipo, mientras que la local depende de que cada persona se acuerde de hacerla.

Pregunta 3

¿Qué riesgo tendría hacer merge a main sin esperar los checks del Pull Request?

Se podría integrar código que rompe pruebas o contiene errores de sintaxis, y ese defecto quedaría en la rama principal, de la que parten todos los integrantes del equipo. El resultado sería que otras personas heredarían un proyecto que no funciona y tendrían que detener su trabajo para descubrir el origen del problema. El PR de esta práctica muestra el valor del check: antes de fusionar, GitHub indicó que todos los checks habían pasado y que no había conflictos.

Pregunta 4

En la ejecución fallida, ¿cuál es la línea esencial del error y qué permitió diagnosticar?

La línea esencial es AssertionError: assert 'CRITICO' == 'NORMAL'.

Esta línea indica que la función devolvió CRITICO y que la prueba esperaba NORMAL. Junto con la referencia tests/test_inventario.py:8 y el nombre test_stock_critico en el resumen del log, permitió concluir que la función cumplía la regla y que el error estaba en el valor esperado de la prueba, que se había modificado a propósito.

Pregunta 5

¿Qué diferencia existe entre un step que usa `uses:` y uno que usa `run:`?

Un step con uses: invoca una *action* reutilizable ya construida, como actions/checkout@v6 o actions/setup-python@v5, que realiza una tarea completa a partir de parámetros. Un step con run: ejecuta directamente comandos de consola dentro del runner, como pip install -r requirements.txt o pytest -v. En resumen, uses: reutiliza una pieza empaquetada y run: ejecuta las instrucciones propias del proyecto.

Pregunta 6

¿Qué información aporta `${{ github.actor }}` en `run-name` y por qué es útil en equipo?

Aporta el nombre del usuario de GitHub que originó la ejecución. En las capturas, cada corrida aparece como «CI ejecutado por smithvza». En un proyecto en equipo esto da trazabilidad: permite saber quién subió el cambio o lanzó el workflow, a quién consultar cuando una ejecución falla y quién participó en las integraciones, lo que también sirve como evidencia de participación individual.

Pregunta 7

Si el workflow pasa pero la aplicación completa falla localmente con Docker, ¿significa que Actions «no sirve»?

No. Actions solo verifica lo que el equipo le pidió: en este caso, la sintaxis de los archivos Python con compileall y el resultado de las pruebas unitarias. No construye imágenes, no levanta contenedores ni comprueba que los servicios se comuniquen entre sí, por lo que una falla de Docker queda fuera de su alcance. Que el workflow pase indica que se cumplen las condiciones mínimas automatizadas, no que todo el sistema integrado funcione. La solución es agregar checks que cubran ese riesgo o validar la aplicación completa con otro mecanismo.

Pregunta 8

Tres checks propuestos para la variante de RapidGo 360 y el error que evita cada uno

Como la regla de negocio exacta depende de la variante asignada al equipo, se proponen tres checks aplicables a cualquier variante, que deben ajustarse a su caso concreto. El primero es ejecutar pytest sobre una regla de negocio propia, con al menos un caso normal, uno límite y uno inválido; evita que un cambio en los umbrales o en la lógica clasifique mal los eventos sin que nadie lo note. El segundo es ejecutar python -m compileall sobre las carpetas Python del proyecto; evita que un error de sintaxis de un integrante llegue a main y bloquee el trabajo de los demás. El tercero es ejecutar docker compose config --quiet; evita subir un archivo de composición con errores de formato o de configuración que solo se descubrirían al intentar levantar los servicios.

Pregunta 9

¿Cuál de esos checks debería implementarse primero y por qué?

Debería implementarse primero el pytest de la regla de negocio, porque es rápido, barato y no requiere levantar servicios como Kafka, y valida directamente la decisión central de la variante. Además, el flujo trabajado en esta práctica demuestra que entrega un resultado claro y diagnosticable cuando algo falla. Los otros dos checks pueden añadirse después con el mismo patrón de steps en el workflow.

Pregunta 10

¿Qué prueba sería demasiado costosa o frágil para ejecutar en cada push y cómo se validaría?

Levantar y mantener en ejecución todo el flujo de datos o dejar el productor enviando datos durante varios minutos en cada push, ya que consumiría muchos minutos del runner, tardaría demasiado y fallaría por factores externos a los cambios del código. Esa validación se haría en un momento específico, como antes de un release o de una entrega, de forma manual en el entorno local con Docker, dejando como evidencia capturas y registros. También podría dejarse en un workflow separado que solo se dispare manualmente con workflow_dispatch, de modo que no se ejecute con cada push.

Conclusiones

La práctica permitió comprobar que un workflow de GitHub Actions es una forma de convertir reglas de calidad en verificaciones repetibles: ante cada push, Pull Request o ejecución manual se descargó el repositorio, se instalaron las dependencias, se validó la sintaxis y se ejecutaron las pruebas. Leer los logs, y no solo el ícono verde o rojo, permitió localizar el step y la línea exacta que explicaron la falla controlada. El flujo rama, Pull Request y checks mostró cómo proteger la rama principal antes de integrar cambios. Estos aprendizajes se trasladarán al proyecto RapidGo 360 definiendo checks ligados a decisiones reales de cada variante.